CHAPTER 5

DIGITAL FILTERS DESIGN

5.1 INTRODUCTION TO DIGITAL FILTERS

A filter is essentially a system or network that selectively changes the wave shape, amplitude-frequency and/or phase-frequency characteristics of a signal in a desired manner. Common filtering objectives are to improve the quality of a signal (for example, to remove or reduce noise), to extract information from signals or to separate two or more signals previously combined to make, for example, efficient use of an available communication channel.

A digital filter, as we shall see later, is a mathematical algorithm implemented in hardware and/or software that operates on a digital input signal to produce a digital output signal for the purpose of achieving a filtering objective. The term digital filter refers to the specific hardware or soft ware routine that performs the filtering algorithm. Digital filters often operate on digitized analogue signals or just numbers, representing some variable, stored in a computer memory.

A simplified block diagram of a real-time digital filter, with analogue input and output signals, is given in Fig. 5.1. The band limited analogue signal is sampled periodically and converted into a series of digital samples, x(n), n = 0, 1, …. The digital processor implements the filtering operation, mapping the input sequence, x(n), into the output sequence, y(n), in accordance with a computational algorithm for the filter. The DAC converts the digitally filtered output into analogue values which are then analogue filtered to smooth and remove unwanted high frequency components.

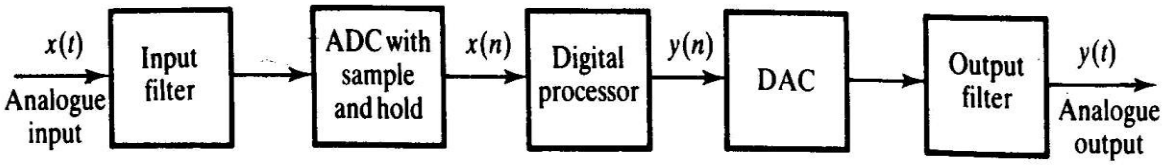


Fig. 5.1 A simplified block diagram of a real-time digital filter with analogue input and output signals.

Digital filters play very important roles in DSP. Compared with analogue filters they are preferred in a number of applications (for example: data compression, biomedical signal processing, speech processing, image processing, data transmission, digital audio, telephone echo cancellation) because of one or more of the following advantages:

1- Digital filters can have characteristics which are not possible with analogue filters, such as a truly linear phase response.

2- Unlike analogue filters, the performance of digital filters does not vary with environmental changes, e.g. thermal variations. This eliminates the need to calibrate periodically.

3- The frequency response of a digital filter can be automatically adjusted if it is implemented using a programmable processor, which is why they are widely used in adaptive filters.

4- Several input signals or channels can be filtered by one digital filter without the need to replicate the hardware.

5- Both filtered and unfiltered data can be saved for further use.

6- Advantage can be readily taken of the tremendous advancements in VLSI technology to fabricate digital filters and to make them small in size, to consume low power, and to keep the cost down.

7- In practice, the precision achievable with analogue filters is restricted; e.g., typically a maximum of only about 60 to 70 dB stopband attenuation is possible

with active filters designed with off-the-shelf components. With digital filters the precision is limited only by the word length used.

8- The performance of digital filters is repeatable from unit to unit.

9- Digital filters can be used at very low frequencies, found in many biomedical applications for example, where the use of analogue filters is impractical. Also, digital filters can be made to work over a wide range of frequencies by a mere change to the sampling frequency.

The following are the main disadvantages of digital filters compared with analogue filters:

1- **Speed limitation.**

The maximum bandwidth of signals that digital filters can handle, in real time, is much lower than for analogue filters. In real-time situations, the analogue-digital filter performance. The conversion time of the ADC and the settling time of the DAC limit the highest frequency that can be processed. Further, the speed of operation of a digital filter depends on the speed of the digital processor used and on the number of arithmetic operations that must be performed for the filtering algorithm, which increases the filter response is made tighter.

2- **Finite word length effects.**

Digital filters are subject to ADC noise resulting from quantizing a continuous signal, and to round off noise incurred during computation. With higher order recursive filters, the accumulation of round off noise could lead to instability.

3- **Long design and development times.**

The design and development times for digital filters, especially hardware development, can be much longer than for analogue filters. However, once developed the hardware and/or software can be used for other filtering or DSP tasks with little or no modifications. Good computer-aided design (CAD) support can make the design of digital filters an enjoyable task, but some expertise is required to make full and effective use of such design aids.

5.2 TYPES OF DIGITAL FILTERS: FIR AND IIR FILTERS

Digital filters are broadly divided into two classes, namely infinite impulse response (IIR) and finite impulse response (FIR) filters. Either type of filter, in its basic form, can be represented by its impulse response sequence, h(k) (k = 0, 1, …), as in Figure 5.2. The input and output signals to the filter are related by the convolution sum, which is given in Eqs. 5.1 for the IIR and in 5.2 for the FIR filter.

$$y(n) = \sum_{k=0}^{\infty} h(k)x(n-k) \tag{5.1}$$

$$y(n) = \sum_{k=0}^{N-1} h(k)x(n-k) \tag{5.2}$$

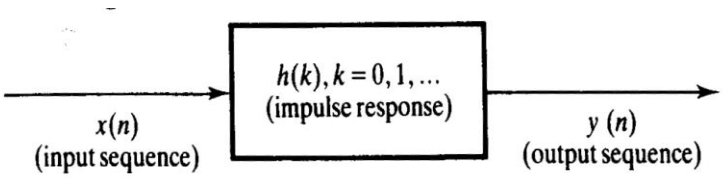


Fig. 5.2 A conceptual representation of a digital filter.

It is evident from these equations that, for IIR filters, the impulse response is of infinite duration whereas for FIR it is of finite duration, since h(k) for the FIR has only N values. In practice, it is not feasible to compute the output of the IIR filter using Eq. 5.1 because the length of its impulse response is too long (infinite in theory). Instead, the IIR filtering equation is expressed in a recursive form:

$$y(n) = \sum_{k=0}^{\infty} h(k)x(n-k) = \sum_{k=0}^{N} a_k x(n-k) - \sum_{k=1}^{M} b_k y(n-k) \tag{5.3}$$

where the a_k and b_k are the coefficients of the filter. Thus, Equations 5.2 and 5.3 are the difference equations for the FIR and IIR filters respectively. These equations, and in particular the values of h(k), for FIR , or a_k and b_k, for IIR, are often very important objectives of most filter design problems. We note that, in Eq. 5.3, the current output sample, y(n), is a function of past outputs as well as present and past input samples, that

is the IIR is a feedback system of some sort. This should be compared with the FIR equation in which the current output sample, y(n), is a function only of past and present values of the input.

Note, however, that when the b_k are set to zero, Eq. 5.3 reduces to the FIR Eq. 5.2.

Alternative representations for the FIR and IIR filters are given in Eqs. 5.4a and 5.4b respectively. These are the transfer functions for these filters and are very useful in evaluating their frequency responses.

$$H(z) = \sum_{k=0}^{N-1} h(k)z^{-k} \tag{5.4a}$$

$$H(z) = \sum_{k=0}^{N} a_k z^{-k} \Big/ \left(1 + \sum_{k=1}^{M} b_k z^{-k} \right) \tag{5.4b}$$

As will become clear in the next few sections, factors that influence the choice of options open to the digital filter designer at each stage of the design process are strongly linked to whether the filter in question is IIR or FIR. Thus, it is very important to appreciate the differences between IIR and FIR, their peculiar characteristics, and more importantly, how to choose between them.

5.3 CHOOSING BETWEEN FIR AND IIR FILTERS

The choice between FIR and IIR filters depends largely on the relative advantages of the two filter types.

1- FIR filters can have an exactly linear phase response. The implication of this is that no phase distortion is introduced into the signal by the filter. This is an important requirement in many applications, for example data transmission, biomedicine, digital audio and image processing. The phase responses of IIR filters are nonlinear, especially at the band edges.

2- FIR filters realized non-recursively, that is by direct evaluation of Eq. 5.2, are always stable. The stability of IIR filters cannot always be guaranteed.

3- The effects of using a limited number of bits to implement filters such as round off noise and coefficient quantization errors are much less sever in FIR than in IIR.

4- FIR requires more coefficients for sharp cutoff filters than IIR. Thus for a given amplitude response specification, more processing time and storage will be required for FIR implementation. However, one can readily take advantage of the computational speed of the FFT and multi-rate techniques to improve significantly the efficiency of FIR implementations.

5- Analogue filters can be readily transformed into equivalent IIR digital filters meeting similar specifications. This is not possible with FIR filters as they have no analogue counterpart. However, with FIR it is easier to synthesize filters of arbitrary frequency responses.

6- In general, FIR is algebraically more difficult to synthesize, if CAD support is not available.

From the above, a broad guideline on when to use FIR or IIR would be as follows:

- Use IIR when the only important requirements are sharp cutoff filters and high throughput, as IIR filters, especially those using elliptic characteristics, will give fewer coefficients than FIR.

- Use FIR if the number of filter coefficients is not too large and, is particular, if little or no phase distortion is desired. One might also add that newer DSP processors have architecture that are tailored to FIR filtering, and indeed some are designed specifically for FIRs.

Example 5.1

The following transfer functions represent two different filters meeting identical amplitude – frequency response specifications:

(1) $$H(z) = \frac{a_0 + a_1 z^{-1} + a_2 z^{-2}}{1 + b_1 z^{-1} + b_2 z^{-2}}$$

Where,

$a_0 = 0.4981819$ $a_1 = 0.9274777$ $a_2 = 0.4981819$

$b_1 = -0.6744878$ $b_2 = -0.3633482$

(2) $$H(z) = \sum_{k=0}^{11} h(k) z^{-k}$$

where,

$h(0) = 0.54603280 \times 10^{-2} = h(11)$

$h(1) = -0.45068750 \times 10^{-1} = h(10)$

$h(2) = 0.69169420 \times 10^{-1} = h(9)$

$h(3) = -0.55384370 \times 10^{-1} = h(8)$

$h(4) = -0.63428410 \times 10^{-1} = h(7)$

$h(5) = 0.57892400 \times 10^{0} = h(6)$

For each filter,

(a) State whether it is an FIR or IIR filter,

(b) Represent the filtering operation in a block diagram form and write down the difference equation, and

(c) Determine and comment on the computational and storage requirements.

Solution

(a) Filters (1) and (2) are IIR and FIR respectively.

(b) The block diagram for filter (1) is given in Fig. 5.3 (a).

The corresponding set of difference equations is:

$$w(n) = x(n) - b_1 w(n-1) - b_2 w(n-2)$$

$$y(n) = a_0 w(n) + a_1 w(n-1) + a_2 w(n-2)$$

The block diagram for filter (2) is given in Fig. 5.3(b). The corresponding difference eq. is:

$$y(n) = \sum_{k=0}^{11} h(k)x(n-k)$$

(c) From examination of the two difference equations the computational and storage requirements for both filters are summarized below:

	FIR	IIR
Number of multiplications	12	5
Number of additions	11	4
Storage locations (coefficients and data)	24	8

It is evident that the IIR filter is more economical in both computational and storage requirements than the FIR filter. However, we could have exploited the symmetry in the FIR coefficients to make the FIR filter more efficient, although at the expense of its obvious implementation simplicity. A point worth making is that, for the same amplitude response specifications, the number of FIR filter coefficients (12 in this example) is typically six times the order (the highest power of z in the denominator) of the IIR transfer function (2 in this case).

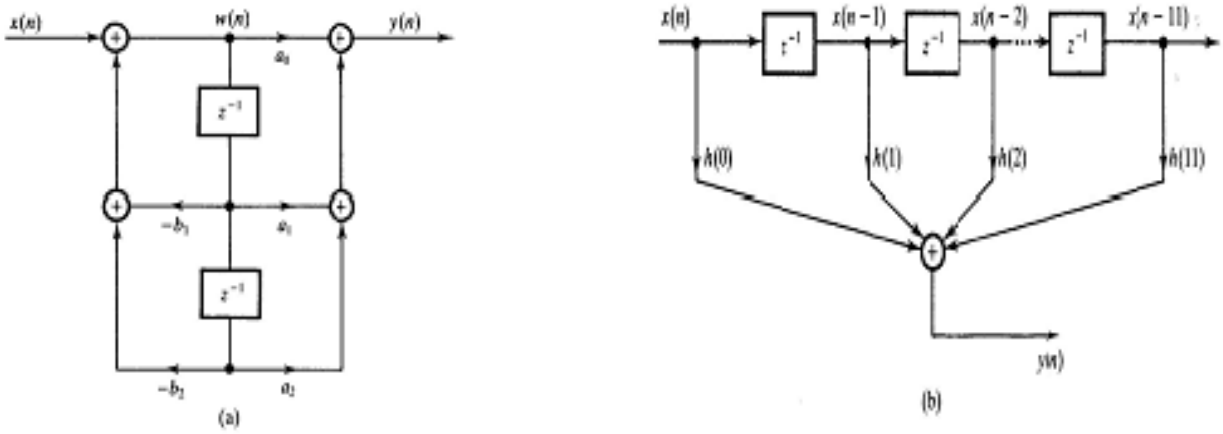


Fig. 5.3 (a) Block diagram representation of the IIR filter of Example 5.1. (b) Block diagram representation of the FIR filter of Example 5.1.

5.4 FILTER DESIGN STEPS

The design of a digital filter involves five steps:

- Specification of the filter requirements;
- Calculation of suitable filter coefficients;
- Representation of the filter by a suitable structure (realization);
- Analysis of the effects of finite word length on filter performance;
- Implementation of filter in software and/or hardware.

The five steps are not necessarily independent; nor are they always performed in the order given. In fact techniques are now available that combine the second and aspects of the third and fourth steps. However, the approach discussed here gives a simple step-by-step guide that will ensure a successful design. To arrive at an efficient filter, it may be necessary to iterate a few times between the steps, especially if the problem specification is not watertight, as is often the case, or if the designer wants to explore other possible designs. Detailed discussions of these steps now follow.

5.4.1 Specification of the filter requirements

Requirement specifications include specifying:

(i) signal characteristics (types of signal source and sink, I/O interface, data rates and width, and highest frequency of interest),

(ii) the characteristics of the filter (the desired amplitude and/or phase responses and their tolerances (if any), the speed of operation and modes of filtering (real time or batch),

(iii) the manner of implementation (for example, as a high level language routine in a computer or as a DSP processor-based system, choice of signal processor), and

(iv) other design constraints (for example, the cost of the filter).

The designer may not have enough information to specify the filter completely at the outset, but as many of the filter requirements as possible should be specified to simplify the design process.

Although the above requirements are application dependent it will be helpful to devote some time on some aspects of (ii). The characteristics of digital filters are often specified in the frequency domain. For frequency selective filters, such as lowpass and bandpass filters, the specifications are often in the form of tolerance schemes. Figure 5.4 depicts such a scheme for a lowpass filter. The shaded horizontal lines indicate the tolerance limits. In the passband, the magnitude response has a peak deviation of δ_P and, in the stopband, it has a maximum deviation of δ_s.

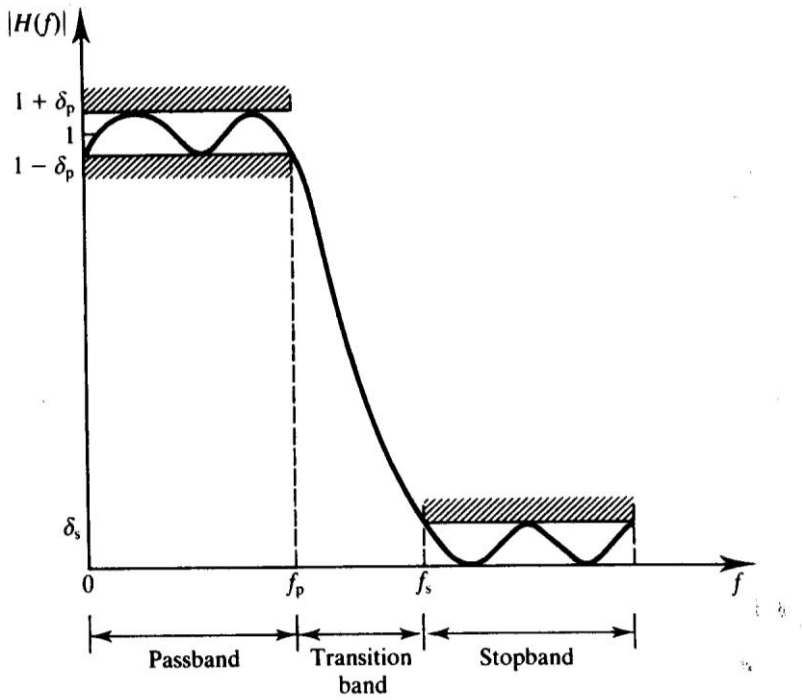


Fig. 5.4 Tolerance scheme for a lowpass filter.

The width of the transition band determines how sharp the filter is. The magnitude response decreases monotonically from the passband to the stopband in this region. The following are the key parameters of interest :

δ_P Passband deviation

δ_s stopband deviation

f_P passband edge frequency

f_s stopband edge frequency

The edge frequencies are often given in normalized form, that is as a fraction of the sampling frequency (f/F_s), but specifications using standard frequency units of hertz, kilohertz are valid and sometimes are more meaningful, especially to the inexperienced designer. Passband and stopband deviations may be expressed as ordinary numbers or in decibels when they specify the passband ripple and minimum stopband attenuation respectively. Thus the minimum stopband attenuation, A_s, and the peak passband ripple, A_P, in decibels are given as (for FIR filters)

$$A_s \text{ (stopband attenuation)} = -20\log_{10} \delta_s \qquad (5.5a)$$

$$A_p \text{ (passband ripple)} = 20 \log_{10}(1+\delta_p) \qquad (5.5b)$$

The phase response of digital filters is often not as meticulously specified as the amplitude response. In many cases it is sufficient to indicate that phase distortion is of concern or that linear phase response is desirable. However, in some applications where filters are used to equalize or compensate a system's phase response, for example, or as phase shifters, then the desired phase response will need to be specified.

Example 5.2

An FIR bandpass filter is to be designed to meet the following frequency response specifications:

passband	0.18 – 0.33 (normalized)
transition width	0.04 (normalized)
stopband deviation	0.001
passband deviation	0.05

1- Sketch the tolerance scheme for the filter.

2- Express the filter band edge frequencies in the standard unit of kilohertz, assuming a sampling frequency of 10kHz, and the stopband and passband deviations in decibels.

Solution

1- The tolerance scheme for the filter is given in Fig. 5.5.

2- The band edge frequencies, at a sampling frequency of 10kHz, and the stopband and passband deviations are given below :

passband	1.8 – 3.3 kHz
stopbands	0 – 1.4 kHz and 3.7 – 5 kHz
stopband attenuation	$-20 \log_{10}(0.001) = 60$ dB
passband ripple	$20 \log_{10}(1 + 0.05) = 0.42$ dB

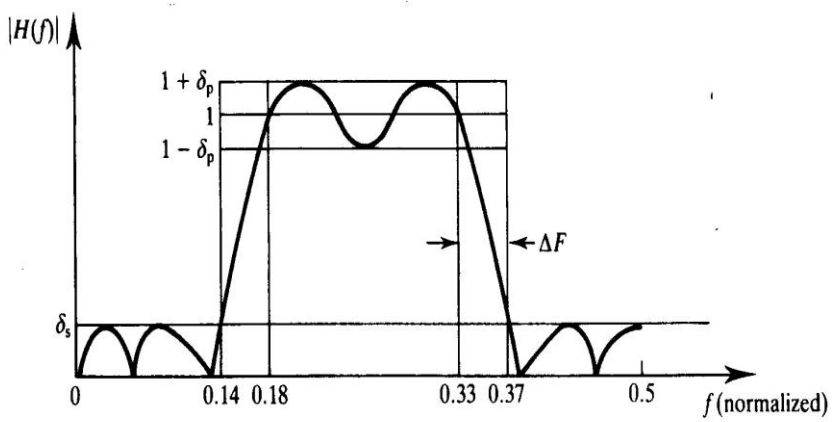


Fig. 5.5 Tolerance scheme for the bandpass filter of Example(2)

5.4.2 Calculation of suitable filter coefficients

In this step, we select one of a number of approximation methods and calculate the values of the coefficients, h(k), for FIR, or a_k and b_k, for IIR, such that the filter characteristics given in Section 5.4.1 are satisfied. The method used to calculate the filter coefficients depends on whether the filter is IIR of FIR type.

Calculations of IIR filter coefficients are traditionally based on the transformation of known analogue filter characteristics into equivalent digital filters. The two basic methods used are the impulse invariant and the bilinear transformation methods. With the impulse invariant method, after digitizing the analogue filter, the impulse response of the original analogue filter is preserved, but not its magnitude–frequency response. Because of inherent aliasing, the method is inappropriate for highpass or bandstop filters. The bilinear method, on the other hand, yields very efficient filters and is well suited to the calculation of coefficients of frequency selective filters. It allows the design of digital filters with known classical characteristics such as Butter-worth, Chebyshev and elliptic. Digital filters resulting from the bilinear transform method will, in general, preserve the magnitude response characteristics of the analogue filter but not the time domain properties. Efficient computer programs now exist for calculating filter coefficients, using the bilinear method, by merely specifying filter parameters of interest. The impulse invariant method is good for simulating analogue systems, but the bilinear method is best for frequency selective IIR filters.

The pole-zero placement method offers an alternative approach to calculating the coefficients of IIR filters. It is an easy way of calculating the coefficients of very simple filters. However, for filters with good amplitude response it is not recommended as it relies on ‘trial and error’ shuffling of the pole and zero positions.

As with IIR filters there are several methods of calculating the coefficients of FIR filters. The three methods discussed in this book in detail are the window, frequency

sampling, and the optimal (Parks – McClellan algorithm). The window method offers a very simple and flexible way of computing FIR filter coefficients, but it does not allow the designer adequate control over the filter parameters. The main attraction of the frequency sampling method is that it allows a recursive realization of FIR filters which can be computationally very efficient. However, it lacks flexibility of an efficient and easy-to-use program, the optimal method is now widely used in industry and, for most applications, will yield the desired FIR filters. Thus, for FIR filters, the optimal method should be the method of first choice unless the particular application dictates other-wise or a CAD facility is unavailable.

In summary, there are several methods of calculating filter coefficients of which the following are the most widely used:

- Impulse invariant (IIR);
- Bilinear transformation (IIR);
- Pole-zero placement (IIR);

- Window (FIR);
- Frequency sampling (FIR);
- Optimal (FIR);

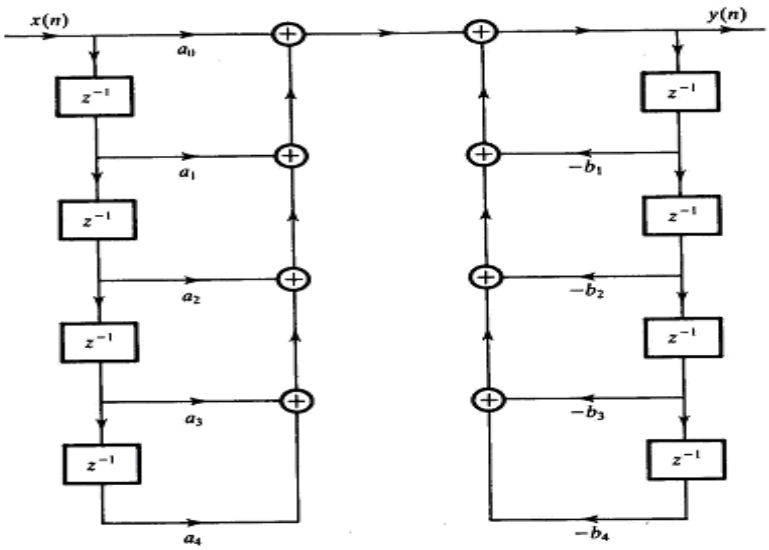
We choose the method that best suits our particular application. Our choice will be influenced by several factors, the most important of which are the critical requirements in the specifications. In general, the crucial choice is really between FIR and IIR. In most cases, if the FIR properties are vital then a good candidate is the optimal method, whereas, if IIR properties are desirable, then the bilinear method will in most cases suffice.

5.4.3 Representation of a filter by a suitable structure (realization)

Realization involves converting a given transfer function, H(z), into a suitable filter structure. Block or flow diagrams are often used to depict filter structures and they show the computational procedure for implementing the digital filter. The structure used depends on whether the filter is an IIR or FIR filter.

For IIR filters, three structures commonly used are the direct form, cascade and parallel forms. The direct form is simply a straightforward representation of the IIR transfer function. In the cascade form, the transfer function of the IIR filter, Equation 5.4b, is factored and expressed as the product of second-order sections. In the parallel form, H(z) is expanded, using partial fractions, as the sum of second-order sections. As an illustration and for comparison, Figure 5.6 shows the block diagrams for a fourth-order (that is, N=4) IIR filter represented in the direct, cascade and parallel structures. The corresponding set of transfer functions and the difference equations describing the filter structures are also given in the figure.

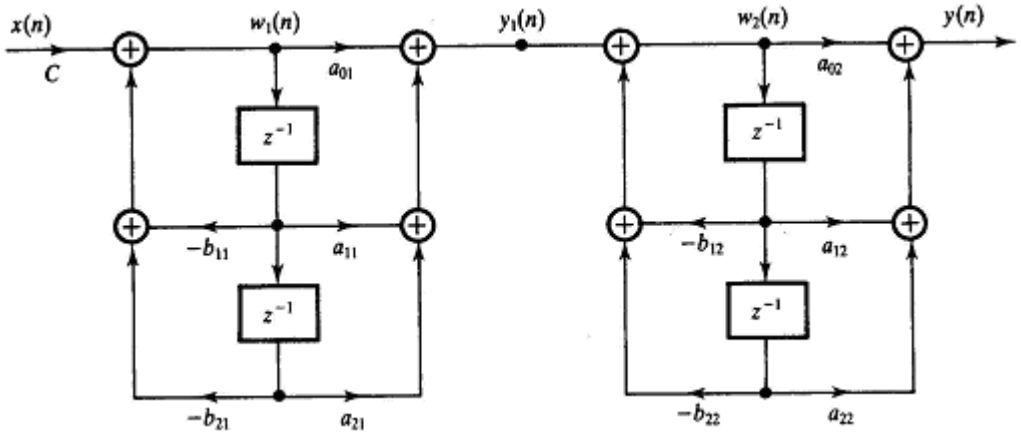
The parallel and cascade structures are the most widely used for IIR because they lead to simpler filtering algorithms and are far less sensitive to the effects of implementing the filter using a finite number of bits than the direct structure. The direct structure suffers sever coefficient sensitivity problems, especially for high order filters, and should be avoided in these cases.



$$H(z)=\frac{\sum_{K=0}^{4} a_k z^{-k}}{1+\sum_{k=0}^{4} bkz^{-k}} \qquad \& \qquad y(n)=\sum_{k=0}^{4} a_k x(n-k)-\sum_{k=1}^{k=4} b_k y(n-k)$$

(a)

Fig. 5.6 (a) Direct realization of a fourth-order IIR filter.

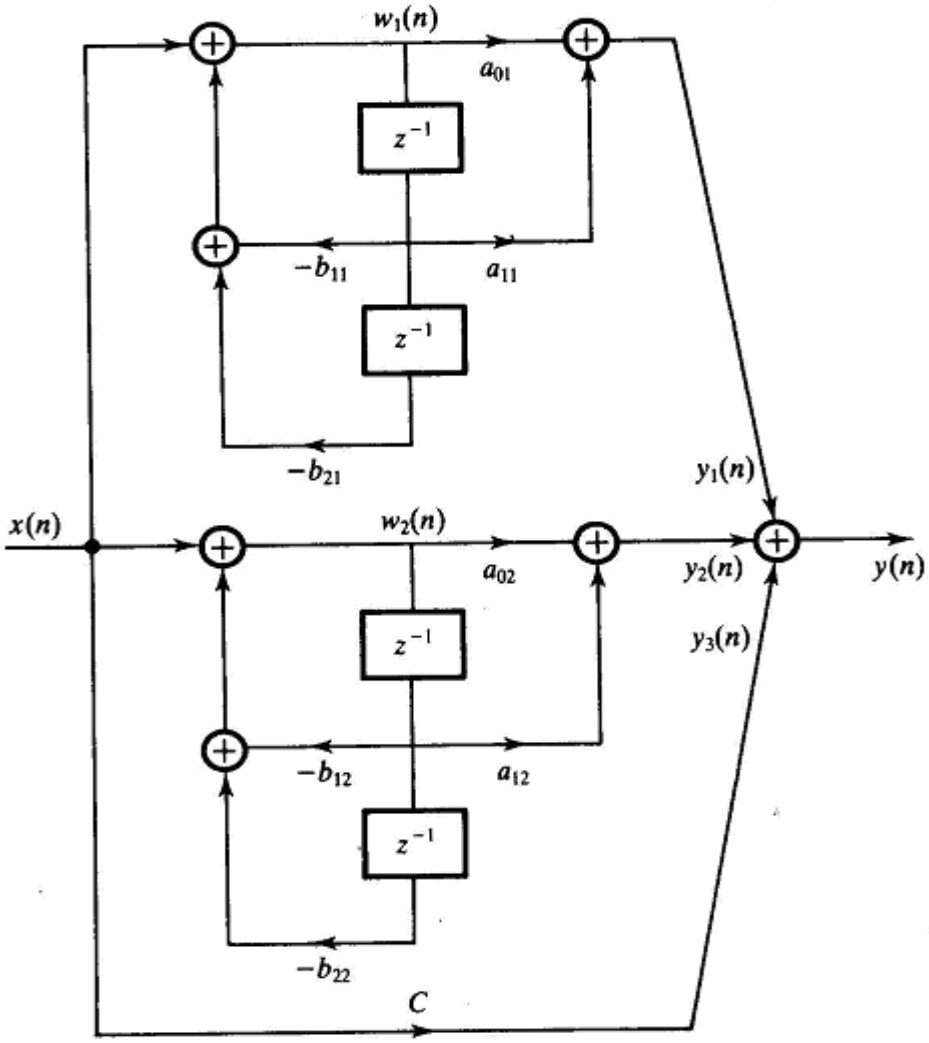


$$H(z)=C\prod_{k=1}^{2}\frac{1+a_{1k}z^{-1}+a_{2k}z^{-2}}{1+b_{1k}z^{-1}+b_{2k}z^{-2}}$$

$$w_1(n)=Cx(n)-b_{11}w_1(n)-b_{21}w_1(n-2)$$
$$y_1(n)=a_{01}w_1(n)+a_{11}w_1(n-1)+a_{21}w_1(n-2)$$
$$w_2(n)=y_1(n)-b_{12}w_2(n-1)-b_{22}w_2(n-2)$$
$$y(n)=a_{02}w_2(n)+a_{12}w_2(n-1)+a_{22}w_2(n-2)$$

(b)

Fig. 5.6 (b) Cascade realization of a fourth-order IIR filter.



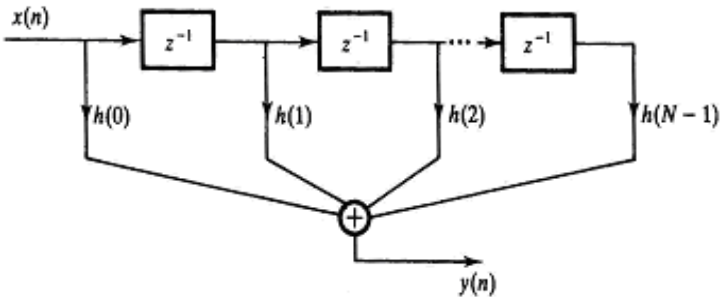
$$H(z) = C + \sum_{k=1}^{2} \frac{a_{0k} + a_{1k} z^{-1}}{1 + b_{1k} z^{-1} + b_{2k} z^{-2}}$$

$$w_1(n) = x(n) - b_{11} w_1(n-1) - b_{21} w_1(n-2)$$
$$w_2(n) = x(n) - b_{12} w_2(n-1) - b_{22} w_2(n-2)$$
$$y_1(n) = a_{01} w_1(n) + a_{11} w_1(n-1)$$
$$y_2(n) = a_{02} w_2(n) + a_{12} w_2(n-2)$$
$$y_3(n) = Cx(n)$$
$$y(n) = y_1(n) + y_2(n) + y_3(n)$$

(c)

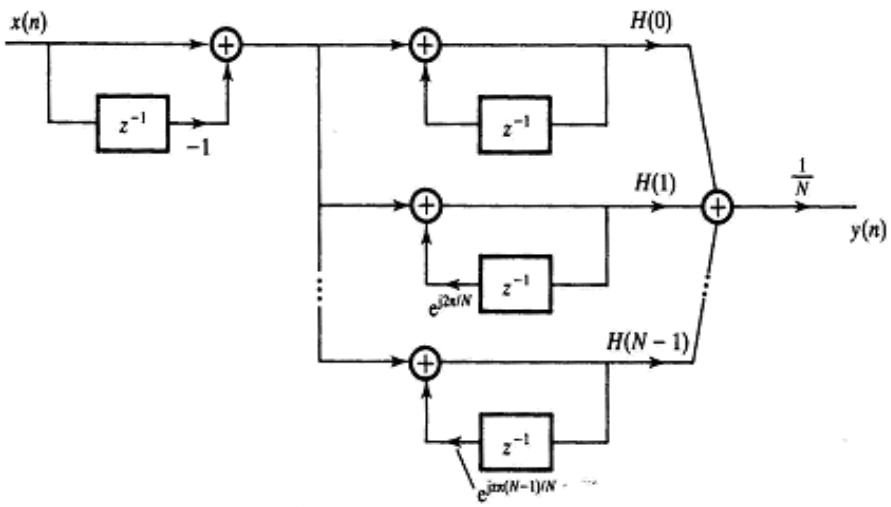
Fig. 5.6 (c) Parallel realization of a fourth-order IIR filter.

The most widely used structure for FIR is the direct form, Fig. 5.7(a), because it is particularly simple to implement. In this form, the FIR is sometimes called a tapped delay line (because it resembles a tapped delay line) or transversal filter. Two other FIR structures that are also used are the frequency sampling structure and the fast convolution technique, Figure 5.7(b) and 5.7(c). compared with the transversal structure, the frequency sampling structure can be computationally more efficient as it leads to fewer coefficients, but it may not be as simple to implement and would require more storage. The fast convolution uses the computational advantage of the fast Fourier transform (FFT) and is particularly attractive in situations where the power spectrum of the signal is also required.



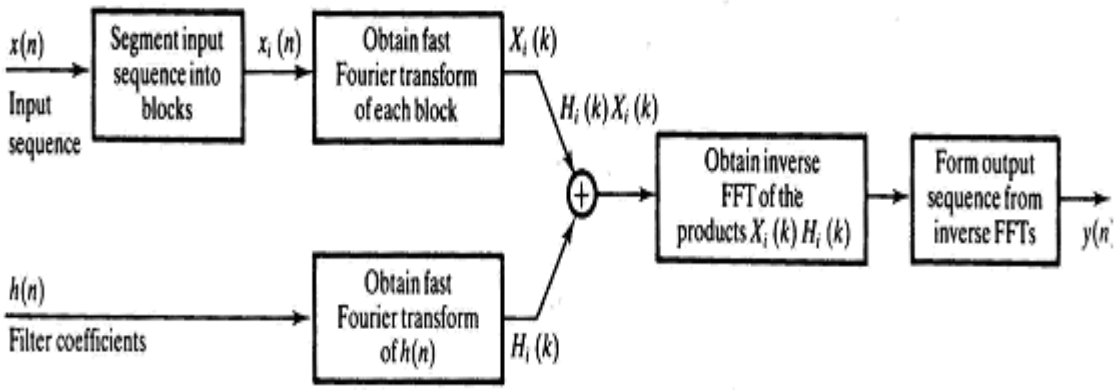
(a)

Fig. 5.7(a) transversal filter (or direct) realization for FIR filters.



(b)

Fig. 5.7(b) frequency sampling realization for FIR filters.

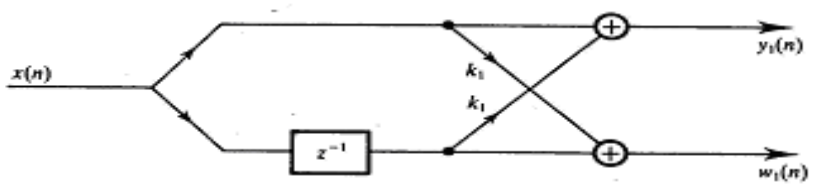


(c)

Fig. 5.7(c) fast convolution realization for FIR filters.

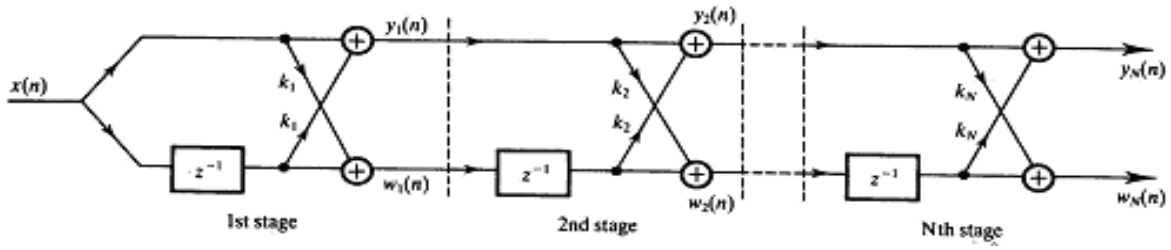
Fig. 5.7 Realization structures for FIR filters: (a) transversal filter (or direct); (b) frequency sampling structure; (c) fast convolution.

There are many other practical structures for digital filters, but most of these are popular only in specific application areas. An example is the lattice structure, which finds use in speech processing and linear prediction applications. The lattice structure may be used to represent FIR as well as IIR filters. The basic lattice structure is characterized by a single input and a pair of outputs, as shown in Figure 5.8(a). A lattice structure, derived from the basic structure in Fig. 5.8(a), for an N-point FIR filter is shown in Fig. 5.8(b), and that for a second-order all-pole IIR filter (that is, with only denominator coefficients) is given in Fig. 5.8(c). Further details of the lattice structure are given in Example (5) section (5.5).



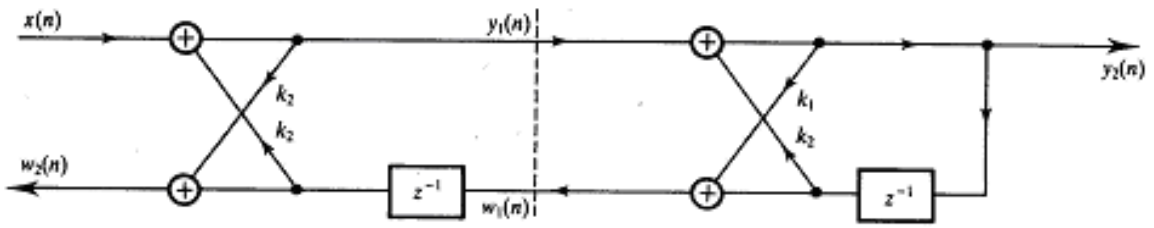
(a)

Fig. 5.8 (a) The basic lattice structure.



(b)

Fig. 5.8 (b) An N-stage FIR lattice structure.



(c)

Fig. 5.8 (c) A 2-stage all-pole IIR lattice structure.

Fig. 5.8 (a) The basic lattice structure. (b) An N-stage FIR lattice structure. (c) A 2-stage all-pole IIR lattice structure.

In summary, the following are the commonly used realization structures for FIR and IIR filters:

FIR	IIR
transversal (direct)	direct form
frequency sampling	cascade
fast convolution	parallel
lattice	

For a given filter the choice between structures depends largely on

(i) whether it is FIR or IIR,

(ii) the ease of implementation and

(iii) how sensitive the structure is to the effects of finite word length.

5.4.4 Analysis of finite word length effects

The approximation and realization steps assume infinite or very high precision. However, in actual implementation it is often necessary to represent the filter coefficients using limited number of bits, typically 8 to 16 bits, and the arithmetic operations indicated in the difference equation are performed using finite precision arithmetic.

The effects of using a finite number of bits are to degrade the performance of the filter and in some cases to make it unusable. The designer must analyze these effects and choose suitable word lengths (that is, numbers of bits), for the filter coefficients, filter variables, that is the input and output samples, and for the arithmetic operations within the filter.

The main sources of performance degradation in digital filters are as follows .

1- Input/output signal quantization.

In particular, the ADC noise due to quantizing of the input signal samples is significant.

2- Coefficient quantization.

This leads to deviations in the frequency response of both FIR and IIR filters, and possibly to instability in IIR filters.

3- Arithmetic round off errors.

The use of finite precision arithmetic to perform filtering operations yields results that require additional bits to represent. When these are quantized to the permissible word length, often by rounding, round off noise is the result. This can cause undesirable effects such as instability in an IIR filter.

4- Overflow.

This occurs when the result of an addition exceeds permissible word length. It leads to wrong output samples and to possible instability in IIR filters.

The extent of filter degradation depends on

(i) the word length and type of arithmetic used to perform the filtering operation,

(ii) the method used to quantize filter coefficients and variables to the chosen word length, and

(iii) the filter structure. From a knowledge of these factors, the designer can assess the effects of finite word length on the filter performance and take remedial action if necessary.

Depending on how the filter is to be implemented, some of the effects may be insignificant. For example, when implemented as a high level language program on most large computers, coefficient quantization and round off errors are not important. For real-time processing, finite word lengths (typically 8 bits, 12 bits, and 16 bits) are used to represent the input and output signals, filter coefficients and the results of arithmetic operations. In these cases, it is nearly always necessary to analyze the effects of quantization on the filter performance.

5.4.5 Implementation of a filter

Having calculated the filter coefficients, chosen a suitable structure, and verified that the filter degradation, after quantizing the coefficients and filter variables to the selected word lengths, is acceptable, the difference equation must be implemented as a soft ware routine or in hardware. Whatever the method of implementation, the output of the filter must be computed, for each sample, in accordance with the difference equation (assuming a time domain implementation).

As the examination of any difference equation will show (Eqs. 5.2 and 5.3), the computation of y(n) (the filter output) involves only multiplications, additions/subtractions, and delays. Thus to implement a filter, we need the following basic building blocks:

- memory (for example ROM) for storing filter coefficients;
- memory (such as RAM) for storing the present and past inputs and outputs, that is {x(n), x(n – 1), ….} and {y(n), y(n – 1),…..};
- hardware or software multiplier(s);
- adder or arithmetic logic unit.

The designer provides these basic blocks and also ensures that they are suitably configured for the application. The manner in which the components are configured depends to a large extent on whether batch (that is, non-real-time) or real-time processing is required. In batch processing, the entire data is already available in some memory device. Such is the case in applications where, for example, experimental data is acquired from elsewhere for later analysis. In this case, the filter is often implemented in a high level language and runs in a general-purpose computer, such as a personal computer or a mainframe computer, where all the basic blocks are already configured. Thus, batch processing may be described as a purely software implementation (although the designer may wish to incorporate additional hardware to increase the speed of processing).

In real-time processing, the filter is required either

(i) to operate on the present input sample, x(n), to produce the current output sample, y(n), before the next input sample arrives, that is within the inter-sample period, or

(ii) to operate on an input block of data, using an FFT algorithm for example, to produce an output block of data within a period proportional to the block length. Real-time filtering may require fast and dedicated hardware if the sample rate is very high or if the filter is of a high order. For most audio frequency work, DSP

processors such as the DSP56000 (by Motorola) and the TMS320C25 (by Texas Instruments) will be adequate and offer considerable flexibility. These processors have all the basic blocks on board, including built-in hardware multiplier(s).

In some applications, standard 8-bit or 16-bit microprocessors such as the Motorola 6800 or 68000 families offer attractive alternative implementations. In addition to the signal processing hardware, the designer must also provide suitable input-output (for example, analogue-digital conversion) interfaces to the digital hardware, depending on the type of data source and sink.

5.5 ILLUSTRATIVE EXAMPLES

Example 5.3

Discuss the five main steps involved in the design of digital filters, using the following design problem to illustrate your answer.

A digital filter is required for real-time physiological noise reduction. The filter should meet the following amplitude response specifications:

passband	0 – 10Hz
stopband	20 – 64Hz
sampling frequency	128Hz
maximum passband deviation	< 0.036 dB
stopband attenuation	> 30dB

Other important requirements are that

1- minimal distortion of the harmonic relationships between the components of the in-band signals is highly desirable,

2- the time available for filtering is limited, the filter being part of a larger process, and

3- the filter will be implemented using the Texas Instruments TMS32010 DSP processor with the analogue input digitized to 12 bits.

Solution

This filter was designed and used in a certain biomedical signal processing project. We shall give here only an outline discussion of the design, postponing detailed discussion to Chapter 6 where FIR filter design methods are fully covered.

1- Requirement specification. As discussed previously, the designer must give the exact role and performance requirements for the filter together with any important constraints. These have already been given for the example.

2- Calculation of suitable coefficients. The requirements of minimal distortion and limited processing time are best achieved with a linear phase FIR filter, with coefficients obtained using the optimal method.

3- Selection of filter structure. The transversal structure will lead to the most efficient implementation using the TMS32010 processor.

4- Analysis of finite word length effects. Since the TMS32010 processor will be used, fixed point arithmetic should be used with each coefficient represented by 16 bits (after rounding) for efficiency. FIR filter degradation may result from input signal quantization, coefficient quantization, round off and overflow errors. A check should be made to ensure that the word lengths are sufficiently long. Analysis of finite word length effects for this case showed that the input quantization noise and deviation in the frequency response due to coefficient quantization are both insignificant. The use of the TMS32010's 32-bit accumulator to sum the coefficient-data products, rounding only the final sum, would reduce round off errors to negligible levels. To avoid

overflow, each coefficient should be divided by $\sum_{k=0}^{N-1}|h(k)|$ before quantizing to 16 bits.

5- Implementation. Design and configure the TMS32010-based hardware (if it does not already exist) with the necessary input/output interfaces. Then write a TMS32010 program to handle the I/O protocols and calculate filter output,

$y(n)= \sum_{k=0}^{N-1} h(k)\, x(n-k)$, for each new input, x(n).

Example 5.4

An analogue filter is to be converted into an equivalent digital filter that will operate at a sampling frequency of 256 Hz. The analogue filter has the transfer function:

$$H(s) = \frac{1}{s^3 + 2s^2 + 2s + 1}$$

1- Obtain suitable coefficients for the digital filter.

2- Assuming that the digital filter is to be realized using the cascade structure, draw a suitable realization block diagram and develop the difference equations.

3- Repeat (2) for the parallel structure.

Solution

1- To preserve the amplitude response of the analogue function, the bilinear method was used to obtain the filter coefficients. The application of the bilinear transformation approach to the analogue transfer function yields the following transfer functions:

$$H(z) = \frac{0.1432(1 + 3z^{-1} + 3z^{-2} + z^{-3})}{1 - 0.1801z^{-1} + 0.3419z^{-2} - 0.0165z^{-3}}$$

2- For the cascade realization, H(z) is factorized using partial fractions :

$$H(z) = 0.1432\frac{1 + 2z^{-1} + z^{-2}}{1 - 0.1307z^{-1} + 0.3355z^{-2}}\frac{1 + z^{-1}}{1 - 0.0490z^{-1}}$$

The block diagram representation and the corresponding set of difference equations are given in Figure 5.9 and as follows:

$w_1(n) = 0.1432x(n) + 0.1307w_1(n-1) - 0.3355w_1(n-2)$
$y_1(n) = w_1(n) + 2w_1(n-1) + w_1(n-2)$
$w_2(n) = y_1(n) + 0.049w_2(n-1)$
$y_2(n) = w_2(n) + w_2(n-1)$

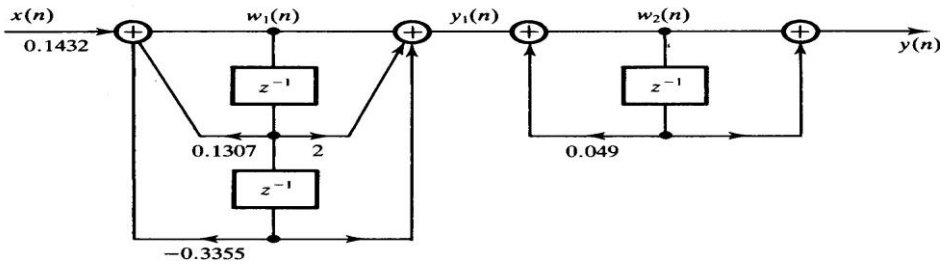


Fig. 5.9

3- For the parallel realization, H(z) is expressed using partial fractions as:

$$H(z) = \frac{1.2916 - 0.08407z^{-1}}{1 - 0.131z^{-1} + 0.3355z^{-2}} + \frac{7.5268}{1 - 0.049z^{-1}} - 8.675$$

The parallel realization diagram and its corresponding set of difference equation are given in Fig. 5.10 and as follows:

$w_1(n) = x(n) + 0.131w_1(n-1) - 0.3355w_1(n-2)$
$y_1(n) = 1.2916w_1(n) - 0.08407w_1(n-1)$
$w_2(n) = x(n) + 0.049w_2(n-1)$
$y_2(n) = 7.5268w_2(n)$
$y_3(n) = -8.6753x(n)$
$y(n) = y_1(n) + y_2(n) + y_3(n)$

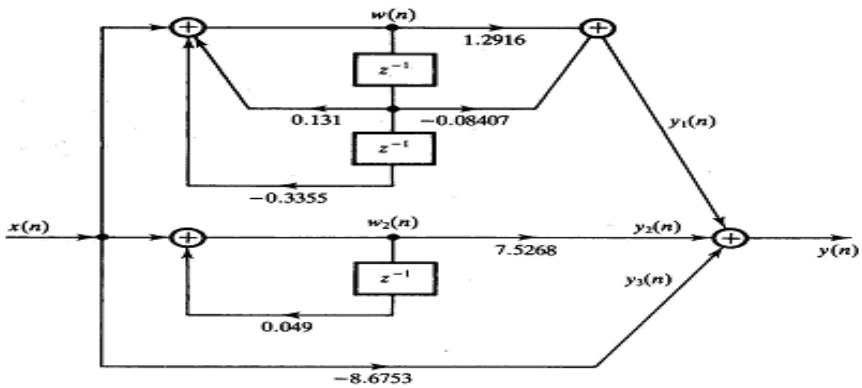


Fig. 5.10

Example 5.5

The transfer function for an FIR filter is given by

$$H(z) = 1 - 1.3435z^{-1} + 0.9025z^{-2}$$

Draw the realization block diagram for each of the following cases:

1- Transversal structure;

2- A two-stage lattice structure.

Calculate the values of the coefficients for the lattice structure.

Solution

1- From the transfer function, the diagram for the transversal structure is given in Figure 5.11. The input and output of the transversal structure are given by

$$y(n) = x(n) + h(1)x(n-1) + h(2)x(n-2) \qquad (5.6)$$

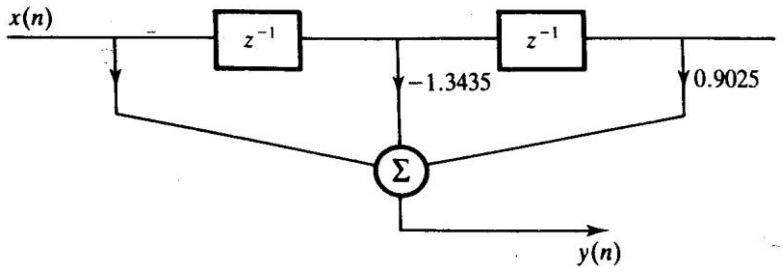


Fig. 5.11

2- A two-stage lattice structure for the filter is given in Figure 5.12. The outputs of the structure are related to the input as:

$$y_2(n) = y_1(n) + K_2w_1(n-1)$$
$$= x(n) + K_1(1 + K_2)x(n-1) + K_2x(n-2) \qquad (5.7a)$$

$$w_2(n) = K_2x(n) + K_1(1 + K_2)x(n-1) + x(n-2) \qquad (5.7b)$$

Comparing Equation 5.6 and 5.7a, and equating coefficients, we have

$$K_1 = \frac{h(1)}{1+h(2)} \qquad \text{and} \qquad K_2 = h(2)$$

from which $K_2 = 0.9025$ and

$K_1 = - 1.3435/ (1 + 0.9025) = - 0.7062.$

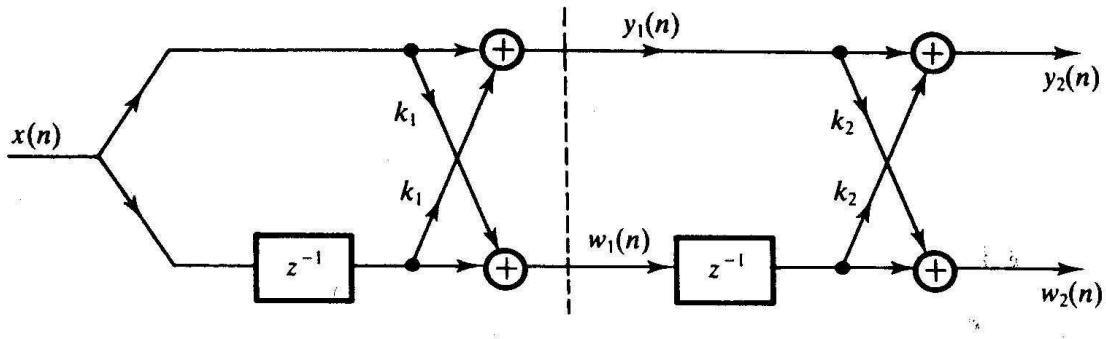


Fig. 5.12

Notice that the coefficients of $y_2(n)$ and $w_2(n)$ (Eqs. 5.7a and 5.7b) are identical except that one is written in reverse order. This is a characteristic feature of the FIR lattice structure. Further details of the lattice structure including recursive techniques for converting the coefficients of an FIR or IIR filter to those of equivalent lattice structures are available in several books (see, for example, Proakis and Manolakis, 1992).

SHEET # 5

5.1 The following transfer functions represent three different filters meeting identical amplitude frequency response specifications:

$$H_1(Z) = \frac{a_o + a_1 Z^{-1} + a_2 Z^{-2}}{1 + b_1 Z^{-1} + b_2 Z^{-2}} \qquad H_2(Z) = \frac{a_3 + a_4 Z^{-1} + a_5 Z^{-2}}{1 + b_3 Z^{-1} + b_4 Z^{-2}}$$, and

$$H_3(Z) = \sum_{k=0}^{22} h_k Z^{-k}$$

Where:

$a_o = 3.136362 \times 10^{-1}$ $a_1 = -5.456657 \times 10^{-2}$ $a_2 = 4.635728 \times 10^{-1}$
$a_3 = -5.456657 \times 10^{-2}$ $a_4 = 3.136362 \times 10^{-1}$
$b_1 = -8.118702 \times 10^{-1}$ $b_2 = 3.339288 \times 10^{-1}$ $b_3 = -2.794577 \times 10^{-1}$
$b_4 = 3.030631 \times 10^{-1}$
and

$h_o = 0.39826480 \times 10^{-1} = h_{22}$
$h_1 = -0.16874380 \times 10^{-1} = h_{21}$
$h_2 = 0.34781130 \times 10^{-1} = h_{20}$
$h_3 = 0.12052890 \times 10^{-1} = h_{19}$
$h_4 = -0.44731860 \times 10^{-1} = h_{18}$
$h_5 = 0.27894610 \times 10^{-1} = h_{17}$
$h_6 = -0.87573360 \times 10^{-1} = h_{16}$
$h_7 = -0.90972060 \times 10^{-1} = h_{15}$
$h_8 = -0.15667550 \times 10^{-1} = h_{14}$
$h_9 = -0.28499560 \times 10^{0} = h_{13}$
$h_{10} = 0.74035030 \times 10^{-1} = h_{12}$
$h_{11} = 0.62349560 \times 10^{-10}$

(a) $H(Z) = H_1(Z) \times H_2(Z)$ (b) $H(Z) = H_1(Z) + H_2(Z)$ (c) $H(Z) = H_3(Z)$
(d) $H(Z) = [\, H_1(Z) \times H_2(Z) \,] + H_3(Z)$ (e) $H(Z) = H_1(Z) + H_2(Z) + H_3(Z)$

For each of the above filters find the following:

1- State whether it is an FIR or IIR filter or neither.

2- Represent the filter operation in a block diagram form an write down the difference equation.

3- Determine and comment on the computational and storage requirements.

5.2 The coefficients of Lattice filter are K_1 = -0.266 and K_2 = 0.69. Draw the realization diagram for the Lattice filter. Compute the impulse response coefficients for the filter and draw the equivalent transversal function.

5.3 A second-order IIR digital filter is characterized by the following transfer function:

$$H(Z) = \frac{1}{1 - 0.9Z^{-1} + 0.81Z^{-2}}$$

Draw the realization diagram for each of the following structure:

(a) Direct.

(b) Lattice.

Then, find the coefficients of the Lattice structure from the given transfer function.

5.4 The coefficients of transversal filter are $h_0 = h_1$ = - 0.266 and h_2 = 0.69. Draw the realization structure for the transversal filter. Find the impulse response and the difference equation of the filter.

Assignment # 5

Give the realization structure of an IIR filter as shown in the following figure. Find the two-step difference equation relating the output, y (n), and input, x (n). Draw the equivalent Lattice filter's structure, and then compute its coefficients.

